AI Fundamentals

A practical guide to Artificial Intelligence concepts

Clear explanations • Visual examples • Real-world analogies

Abir GHARBI

STMicroelectronics — Support & Maintenance Team — 2026

Contents

1 What is Artificial Intelligence? 4
- 1.1 The Simple Definition . . . 4
- 1.2 AI is NOT Magic . . . 4
- 1.3 Types of AI . . . 4
- 1.4 The AI Family Tree . . . 5

2 Machine Learning — When Computers Learn from Data 6
- 2.1 The Fundamental Idea . . . 6
- 2.2 What Makes ML Different: Statistics + Adaptation . . . 6
- 2.3 The Role of Dataset Size . . . 7
- 2.4 The Three Types of Machine Learning . . . 7
  - 2.4.1 Supervised Learning — Learning with a Teacher . . . 7
  - 2.4.2 Unsupervised Learning — Learning without Labels . . . 7
  - 2.4.3 Reinforcement Learning — Learning by Trial and Error . . . 7
- 2.5 ML Algorithms — The Toolbox . . . 8

3 Deep Learning & Neural Networks — ML Goes Deeper 9
- 3.1 From ML to Deep Learning: The Natural Next Step . . . 9
- 3.2 What is a Neural Network? . . . 9
- 3.3 What Makes it "Deep"? . . . 10
- 3.4 Why Deep Learning Exploded in 2012 . . . 10
- 3.5 What Deep Learning Made Possible . . . 10

4 How to Train a Model — The Mechanics 11
- 4.1 The Big Picture: What Does "Training" Mean? . . . 11
- 4.2 The Training Lifecycle . . . 11
  - 4.2.1 Step 1: Collect Data . . . 11
  - 4.2.2 Step 2: Prepare Data . . . 11
  - 4.2.3 Step 3: Split Data into Three Sets . . . 12
  - 4.2.4 Step 4: Train the Model . . . 12
  - 4.2.5 Step 5: Evaluate . . . 13
  - 4.2.6 Step 6: Iterate . . . 13
  - 4.2.7 Steps 7–8: Deploy & Monitor . . . 13
- 4.3 What the Model Learns: Weights . . . 13
- 4.4 Overfitting vs. Underfitting . . . 14
- 4.5 Training vs. RAG — Two Different Approaches . . . 14

5 Natural Language Processing — Teaching Machines to Read 15
- 5.1 The Bridge: Deep Learning Meets Language . . . 15
- 5.2 Why Language is Hard for Machines . . . 15

5.3 Key NLP Tasks . . . 15
5.4 Tokenization: The First Step . . . 15
5.5 TF-IDF: Classic Word Importance . . . 16
5.6 The Evolution of NLP . . . 16

6 The Transformer Revolution — How GPT Was Born 17
6.1 The Problem Before Transformers . . . 17
6.2 The Breakthrough: Attention (2017) . . . 17
6.3 The Birth of GPT (2018) . . . 18
6.4 The GPT Timeline — A Story of Scale . . . 18
6.5 Why "Predict the Next Word" is So Powerful . . . 18
6.6 BERT vs. GPT — Two Transformer Strategies . . . 19

7 Generative AI — Machines That Create 20
7.1 What is Generative AI? . . . 20
7.2 How Generative AI Works (Simplified) . . . 20
7.3 The Generative AI Ecosystem (2024–2026) . . . 21
7.4 Why Generative AI Changed Everything . . . 21

8 Large Language Models — The Engines of Modern AI 22
8.1 What is an LLM? . . . 22
8.2 How LLMs Generate Text . . . 22
8.3 The Hallucination Problem . . . 22
8.4 Key LLM Concepts . . . 23
8.5 The LLM Landscape (2024–2026) . . . 23
8.6 LLM Limitations — What They Cannot Do . . . 23

9 Embeddings — How AI Understands Meaning 25
9.1 The Problem: Computers Only Understand Numbers . . . 25
9.2 What is an Embedding? . . . 25
9.3 Why Embeddings Matter for Search . . . 25
9.4 Cosine Similarity . . . 25
9.5 Embeddings Are the Foundation of RAG . . . 26

10 Specialized Chatbots — Why Generic AI Isn't Enough 27
10.1 Generalist vs. Specialized . . . 27
10.2 Why Specialize? . . . 27
10.3 Three Approaches to Specialization . . . 27
10.4 Why We Chose RAG . . . 28

11 RAG — Retrieval-Augmented Generation 29
11.1 The Problem RAG Solves . . . 29
11.2 How RAG Works — Step by Step . . . 29
11.3 Why Data Quality is CRITICAL for RAG . . . 30
11.4 Chunking Strategies . . . 30

12 Feature Engineering — Making Data Useful 31
12.1 What is Feature Engineering? . . . 31
12.2 Why It's Often More Impactful Than the Algorithm . . . 31
12.3 Why It's Critical for RAG . . . 31

13 Agentic AI — Autonomous Intelligent Agents 32
13.1 What is an AI Agent? . . . 32

13.2 Components of an Agent . . . 32
13.3 Types of Agents . . . 32
13.4 In Our Project: The Image Enrichment Agent . . . 32
13.5 In Our Project: The Preprocessing Pipeline Agent . . . 33

14 Vector Databases — Where Meaning is Stored **35**
14.1 What is a Vector Database? . . . 35
14.2 How Vector Search Works . . . 35
14.3 The Landscape . . . 35

15 Computer Vision — When AI Sees **36**
15.1 What is Computer Vision? . . . 36
15.2 Key Tasks . . . 36

16 Evaluation — Measuring AI Quality **37**
16.1 Why Evaluation Matters . . . 37
16.2 Key Metrics for RAG . . . 37
16.3 Offline vs. Online Evaluation . . . 37
16.4 The Iteration Loop . . . 37

17 Data Generation — Creating New Knowledge **39**
17.1 What is Data Generation? . . . 39
17.2 Why Generate Data? . . . 39
17.3 Types of Generated Data . . . 39
17.4 Derived Data: The Most Valuable Pattern . . . 40
17.5 Quality Control . . . 40

18 Advantages, Limits, and Challenges of AI **41**
18.1 Advantages . . . 41
18.2 Limits . . . 41
18.3 Ethical Considerations . . . 41

19 Glossary **42**

20 Conclusion **44**

Chapter 1

What is Artificial Intelligence?

1.1 The Simple Definition

Artificial Intelligence (AI) is the science of building computer systems that can perform tasks normally requiring human intelligence: understanding language, recognizing images, making decisions, and learning from experience.

Analogy

Think of AI as a very fast, very obedient assistant. It never gets tired, can read thousands of documents in seconds, remembers everything perfectly — but has **zero** common sense and only does what it was trained to do. It's a powerful tool, not a thinking being.

1.2 AI is NOT Magic

This is the most important point to start with. AI systems are **software** built on mathematics and statistics. They do not think, feel, or understand. They process data, find patterns, and produce outputs based on those patterns.

An AI system is only as good as:

- The **data** it was trained on (garbage in = garbage out)
- The **algorithms** humans chose for it
- The **evaluation** humans apply to verify it works

1.3 Types of AI

Type	What it does	Exists today?
Narrow AI	One specific task, does it very well	Yes (all current AI)
General AI	Any intellectual task like a human	No — does not exist yet
Super AI	Surpasses all human intelligence	Science fiction

Every AI system you interact with today — ChatGPT, Google Translate, Siri, our STM32 chatbot — is Narrow AI. It excels at one thing but cannot do everything.

1.4 The AI Family Tree

Understanding AI requires seeing how different fields fit together. Think of it as a set of Russian nesting dolls:

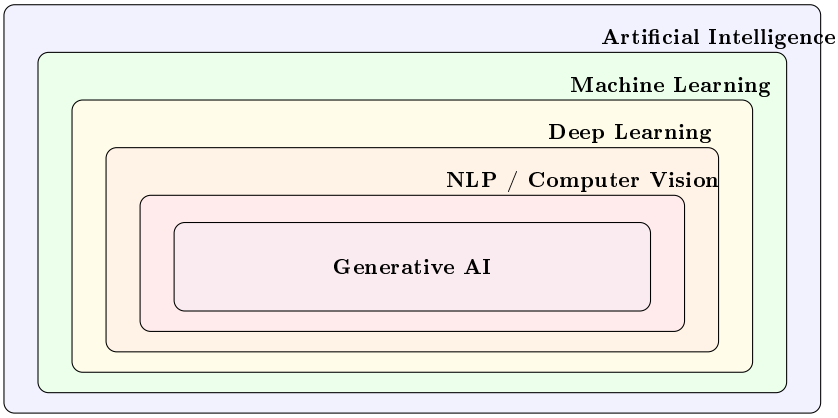


Figure 1.1: The AI family tree: each layer is a specialization of the previous one.

The rest of this document follows this nesting: we start from the outside (AI, ML) and dive deeper until we reach Generative AI and LLMs.

Chapter 2

Machine Learning — When Computers Learn from Data

2.1 The Fundamental Idea

Machine Learning (ML) is where the story really begins. It is the subset of AI where systems **learn from data** instead of being explicitly programmed with rules.

Traditional programming	**Machine Learning**
Human writes the rules	System discovers the rules
"IF email contains 'free money' THEN spam"	Show 10,000 labeled emails, system learns what spam looks like
Rules are brittle — break on new patterns	Adapts to new patterns automatically

Analogy

The child learning fruits. A child doesn't need a rulebook that says "a fruit is round, colorful, grows on trees." Instead, you show the child 100 fruits and say "this is an apple, this is a banana, this is a grape." After enough examples, the child recognizes new fruits they've never seen before. **That's ML**: learning from examples, not from rules.

2.2 What Makes ML Different: Statistics + Adaptation

At its core, ML is **applied statistics at scale**. The system:

1. Looks at thousands of examples
2. Finds **statistical patterns** (correlations, frequencies, clusters)
3. Builds a **mathematical model** that captures those patterns
4. Uses that model to make **predictions** on new, unseen data

The key word is ADAPTATION: unlike a fixed program, an ML system **improves** when you give it more data. More examples = better pattern detection = better predictions.

2.3 The Role of Dataset Size

Dataset size matters enormously — but differently depending on the problem:

Dataset size	What's possible	Limitations
$<$ 100 examples	Simple rules, basic statistics	Model cannot generalize
100 – 10,000	Traditional ML works well	Need careful feature engineering
10,000 – 1M	Deep Learning becomes viable	Need significant compute
$>$ 1 Billion	Large Language Models	Need massive GPU clusters

Analogy

Learning to cook. With 5 recipes, you can follow instructions. With 500 recipes, you start seeing patterns ("most Italian dishes use olive oil and garlic"). With 50,000 recipes from all cultures, you understand cooking deeply enough to *invent* new dishes. Dataset size determines how deep the learning goes.

2.4 The Three Types of Machine Learning

2.4.1 Supervised Learning — Learning with a Teacher

The system learns from **labeled examples**: input/output pairs where a human has provided the "correct answer."

- **Classification**: assign a category (spam vs. not spam, bug vs. feature request)
- **Regression**: predict a number (temperature tomorrow, house price)

Analogy

Like a student with an answer key. They study 1,000 solved exercises, then take an exam with new exercises. The more solved exercises they studied, the better they'll do on the exam.

2.4.2 Unsupervised Learning — Learning without Labels

The system finds patterns in data **without being told what to look for**. Nobody provides correct answers.

- **Clustering**: group similar items ("these 500 issues are about the same bug")
- **Dimensionality reduction**: simplify complex data while keeping the important stuff

Analogy

Like sorting a pile of 1,000 foreign-language books into groups. You don't understand the text, but you can group them by cover color, size, thickness, and font style. You've found structure without anyone telling you what the "right" groups are.

2.4.3 Reinforcement Learning — Learning by Trial and Error

The system learns by **experimenting**: it takes actions, receives rewards or penalties, and gradually learns which actions lead to good outcomes.

- Used in: game AI (AlphaGo), robotics, self-driving cars

Analogy

Like training a dog: reward for sitting, nothing for jumping. After 100 attempts, the dog always sits because it has learned which action gets treats.

2.5 ML Algorithms — The Toolbox

Different problems need different tools. Here's a simplified view:

Algorithm family	How it works (simplified)	Good for
Linear models	Draw a straight line between categories	Simple classification
Decision trees	Ask yes/no questions in sequence	Tabular data
Random forests	Many trees voting together	Most structured data problems
Neural networks	Layers of connected neurons	Complex patterns (language, images)
k-NN	"Look at the k most similar examples"	Recommendation, similarity

The takeaway: you don't need to know the math. Just understand that ML = showing examples to a statistical system that learns patterns and makes predictions. The choice of algorithm depends on the data and the problem.

Chapter 3

Deep Learning & Neural Networks — ML Goes Deeper

3.1 From ML to Deep Learning: The Natural Next Step

We just saw that Machine Learning finds patterns in data. But traditional ML has a limitation: for complex problems (understanding language, recognizing faces, generating images), **you need to manually engineer the right features**. Someone has to decide WHAT information is useful.

Deep Learning removes this limitation. It automatically discovers what features are important — learning not just the patterns, but also **how to represent the data** in the first place.

Analogy

Traditional ML = You tell an artist "paint a sunset with orange, pink, and gold colors" (you specify the features).
Deep Learning = You show an artist 10,000 sunset photos and say "learn to paint sunsets." The artist figures out which colors, shapes, and compositions matter — on their own.

3.2 What is a Neural Network?

A neural network is inspired by the human brain (very loosely). It consists of layers of connected "neurons" that process information step by step.

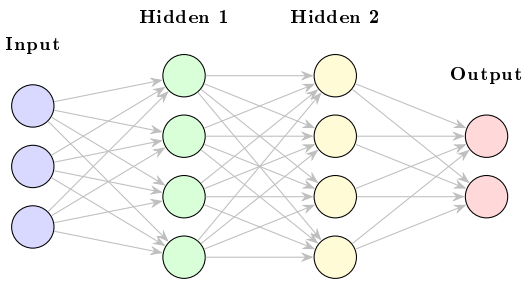


Figure 3.1: A neural network with 2 hidden layers. Each line is a "weight" — a number adjusted during training.

- **Input layer**: receives the raw data (text, image pixels, numbers)
- **Hidden layers**: progressively transform and abstract the data — each layer detects increasingly complex patterns
- **Output layer**: produces the final result (a classification, a prediction, a generated word)

Each connection between neurons has a **weight** — a number that gets adjusted during training. Training a neural network = finding the millions of weights that produce correct outputs.

3.3 What Makes it "Deep"?

"Deep" simply means **many layers**. A network with 2 layers is "shallow." A network with 100+ layers is "deep." More layers = the ability to learn more complex, abstract patterns.

Type	Layers	Capable of
Traditional ML	0 (no layers)	Simple patterns, needs hand-crafted features
Shallow network	1–3 layers	Basic pattern recognition
Deep network	10–100+	Language understanding, image recognition
Very deep (GPT-4)	120+ layers	Complex reasoning, generation, translation

3.4 Why Deep Learning Exploded in 2012

Deep Learning concepts existed since the 1980s. So why did they only become powerful recently? Three things came together:

1. **Massive data** — the internet produced billions of texts, images, videos
2. **Powerful GPUs** — graphics cards designed for gaming turned out to be perfect for neural network math
3. **Better algorithms** — new training techniques (dropout, batch normalization, attention) that prevent the network from getting stuck

Analogy

Deep Learning is like a rocket engine that existed on paper for decades. It only flew when three things appeared simultaneously: enough fuel (data), a powerful enough engine (GPUs), and better aerodynamics (algorithms). Without all three, the rocket stays on the ground.

3.5 What Deep Learning Made Possible

- **Language understanding** — chatbots, translation, summarization (→ leads to NLP and LLMs)
- **Image recognition** — face unlock, medical imaging, autonomous driving
- **Speech recognition** — Siri, Alexa, Google Assistant
- **Content generation** — writing text, creating images, composing music

The next chapter shows what happened when Deep Learning was applied specifically to language...

Chapter 4

How to Train a Model — The Mechanics

4.1 The Big Picture: What Does "Training" Mean?

Now that you understand what ML and neural networks are, let's look at **how** they actually learn.

When we say "training a model," we mean: **showing it thousands of examples so it discovers the patterns by itself.**

The model starts knowing **nothing**. Through training, it gradually learns what matters and what doesn't.

Analogy

Learning to recognize cats. Show a child 10,000 photos: some with cats, some without. At first, the child guesses randomly. But after thousands of examples with the label "cat" or "not cat," the child notices: pointed ears, whiskers, fur, small nose... The child *discovered* the rules from examples — nobody listed them. **That's exactly how training works.**

4.2 The Training Lifecycle

Training is not a single action — it's a **cycle**. If one step is weak, everything suffers.

4.2.1 Step 1: Collect Data

You need **examples** of the problem you want to solve. More examples, and more diverse examples = better learning.

Critical rule: examples must be **representative** of real-world situations. Training only on English emails? The model will fail on French emails.

4.2.2 Step 2: Prepare Data

Raw data is never ready. You must:

1. **Clean**: remove noise, fix errors, normalize formats
2. **Engineer features**: extract the useful signals (more in Chapter 12)
3. **Label**: tag each example with the correct answer

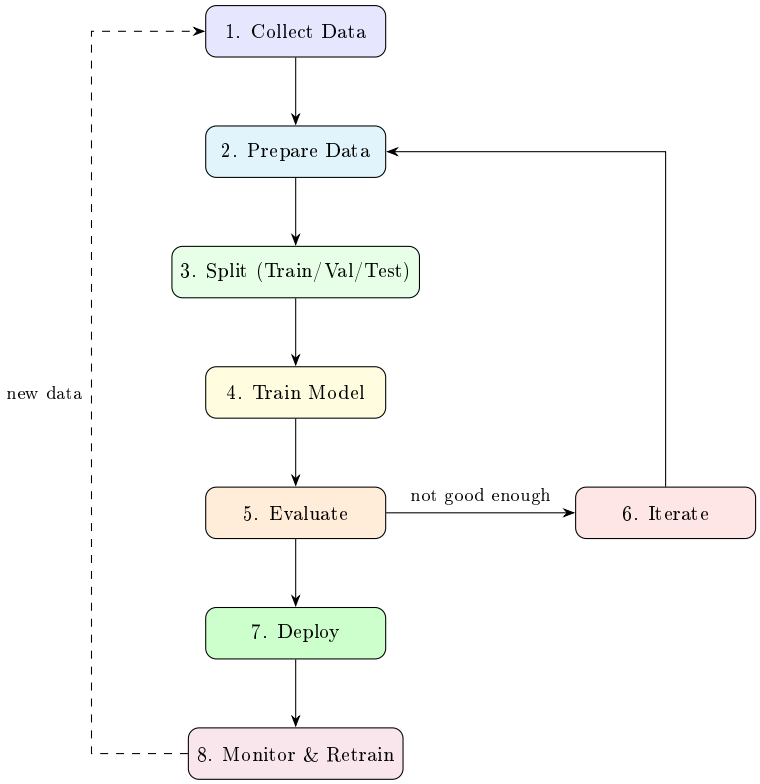


Figure 4.1: The model training lifecycle: an iterative process with feedback loops.

Analogy

Kitchen prep. Before cooking, you wash vegetables, peel them, cut them to size, remove bad parts. Data preparation is the kitchen prep of AI — skip it, and the dish tastes terrible no matter how good the recipe.

4.2.3 Step 3: Split Data into Three Sets

Set	Size	Purpose
Training set	∼80%	Model **learns** from these
Validation set	∼10%	Check progress **during** training
Test set	∼10%	Final measurement — used **once**

Why? If you test on the same data you trained on, it's like a student who memorizes exact exam questions — they'll score 100% but fail any new question.

Analogy

Driving school. Training set = practice drives. Validation set = mock exams. Test set = the real driving test on a route you've never driven. Pass that, and you can truly drive.

4.2.4 Step 4: Train the Model

The actual learning happens in a loop:

1. Take one example
2. Make a prediction (initially random)

3. Compare to the correct answer
4. Calculate the **error** (how far off)
5. Adjust internal weights slightly to reduce that error
6. Repeat thousands/millions of times

One full pass through all training data = one **epoch**. Models train for 10 to 1,000+ epochs.

Analogy

Darts. First throw misses entirely. Feedback: "too far left." Adjust. Next: "a bit high." Adjust. After 1,000 throws, you consistently hit near the bullseye. Each throw = one training step. Feedback = the error. Your muscle adjustment = the weight update.

4.2.5 Step 5: Evaluate

Measure performance on data the model has **never seen**:

- **Accuracy**: % of correct predictions
- **Precision**: when it says "yes," how often is it right?
- **Recall**: of all actual positives, how many did it catch?

4.2.6 Step 6: Iterate

Not good enough? Don't throw everything away — adjust:

- More/better data → back to step 1
- Better features → back to step 2
- Different algorithm → change model

4.2.7 Steps 7–8: Deploy & Monitor

Deploy to production. But models **degrade over time** (the world changes — new patterns appear). This is **model drift**. Solution: retrain periodically with fresh data.

4.3 What the Model Learns: Weights

During training, the model adjusts millions of internal numbers called **weights**. GPT-4 has ∼1.8 **trillion** weights.

After training, weights are **frozen** — the model stops learning and just applies what it knows.

Analogy

Mixing board. A studio with 175 billion knobs. Training = turning every knob until the output sounds perfect. Once set, the studio produces great music for any song. The knob positions = the model's weights.

Problem	What happens	Analogy
Overfitting	Memorizes training data, fails on new data	Student memorizes exact answers, can't solve variations
Underfitting	Too simple, misses patterns	Student didn't study enough, guesses randomly
Good fit	Learns patterns, generalizes well	Student understood concepts, solves new problems

	Training a model	Using RAG
What you do	Teach the model (changes weights)	Give it documents to read at query time
Analogy	Teaching a language over months	Giving a dictionary to look up words
Time	Hours to weeks	Minutes to hours
Cost	Very expensive (GPU clusters)	Moderate
Updates	Retrain = slow & expensive	Update docs = fast & cheap

4.4 Overfitting vs. Underfitting

4.5 Training vs. RAG — Two Different Approaches

We don't train the LLM — we optimize the DATA it retrieves. The LLM is a "rented brain"; we make sure it has the right documents to read.

Chapter 5

Natural Language Processing — Teaching Machines to Read

5.1 The Bridge: Deep Learning Meets Language

We saw that Deep Learning excels at discovering patterns. When researchers applied DL to **text data**, an entire field was revolutionized: **Natural Language Processing** (NLP).

NLP is the branch of AI that enables computers to understand, interpret, and generate human language.

Analogy

Imagine you land in a country where you don't speak the language. NLP is like a universal translator chip in your brain — it converts any text or speech into something you understand, and lets you respond in that language. For computers, text is naturally a foreign language (they only understand numbers). NLP bridges that gap.

5.2 Why Language is Hard for Machines

- **Ambiguity**: "bank" = financial institution or river bank?
- **Context**: "I saw her duck" = she ducked, or she owns a duck?
- **Sarcasm/Irony**: "Oh great, another meeting" = not actually great
- **Coreference**: "John told Peter that *he* was wrong" — who is "he"?
- **World knowledge**: understanding requires knowing things about the world

5.3 Key NLP Tasks

5.4 Tokenization: The First Step

Before any NLP system can process text, the text must be split into smaller units called **tokens**. This is always step 1.

```
"HAL_UART_Init() failed on STM32H7"
  -> ["HAL", "_", "UART", "_", "Init", "()", "failed", "on", "STM32", "H7"]
```

Task	What it does	Example
Tokenization	Split text into units	"Hello world" → ["Hello", "world"]
NER	Identify entities	"STM32H7 has a bug" → STM32H7=MCU
Sentiment Analysis	Detect emotion	"Works perfectly!" → positive
Classification	Assign categories	Email → spam / not spam
Summarization	Condense text	10-page report → 3 sentences
Translation	Convert between languages	English → French
Question Answering	Extract answers	"Capital of France?" → "Paris"

Modern LLMs use **subword tokenization** (BPE): common words stay whole, rare words are split into pieces. This way the model can handle any word — even ones it has never seen.

5.5 TF-IDF: Classic Word Importance

Before Deep Learning dominated NLP, the standard technique was TF-IDF:

- **TF** = how often a word appears in THIS document
- **IDF** = how rare the word is across ALL documents
- **TF-IDF** = **TF** × **IDF** = high score means the word is frequent here but rare elsewhere (= highly discriminating)

Limitation: TF-IDF treats words as independent units with no concept of meaning. "UART error" and "USART failure" would get zero similarity even though they mean the same thing. This is why embeddings (Chapter 9) replaced TF-IDF for semantic tasks.

5.6 The Evolution of NLP

Era	Period	Approach
Rule-based	1950s–1990s	Hand-written grammar rules
Statistical	1990s–2010s	Probabilities from large corpora (TF-IDF, n-grams)
Deep Learning	2013–2017	Neural networks (RNNs, LSTMs)
Transformers	**2017–now**	**Attention mechanism → GPT, BERT...**

The last row — Transformers — changed everything. That's the subject of the next chapter.

Chapter 6

The Transformer Revolution — How GPT Was Born

6.1 The Problem Before Transformers

Before 2017, NLP models (RNNs, LSTMs) processed text **word by word, left to right**. This had two major problems:

1. **Slow**: they couldn't process words in parallel — each word had to wait for the previous one
2. **Forgetful**: for long texts, the model "forgot" what was at the beginning by the time it reached the end

Analogy

Imagine reading a 500-page novel but you can only remember the last 50 pages. By the time you reach the ending, you've forgotten the beginning. Old NLP models had this exact problem with long texts.

6.2 The Breakthrough: Attention (2017)

In 2017, researchers at Google published a paper titled *"Attention Is All You Need."* It introduced the **Transformer** architecture.

The key idea: **Attention** — the model can look at ALL words in a text simultaneously and decide which ones are most relevant to each other.

- Instead of reading left-to-right, the model sees the ENTIRE text at once
- For each word, it calculates how much "attention" to pay to every other word
- This allows understanding long-range relationships ("the cat that I saw yesterday at the park near the river" — "cat" connects to "saw" even though 8 words separate them)

Analogy

Old models = reading a sentence with a tiny flashlight that only illuminates one word at a time.
Transformers = reading the entire page at once under full light, with the ability to draw arrows between related words no matter how far apart they are.

6.3 The Birth of GPT (2018)

OpenAI took the Transformer architecture and asked: "What if we train a massive Transformer on ALL the text on the internet, with one simple task: predict the next word?"

The result was **GPT** (Generative Pre-trained Transformer):

- **Generative**: it generates text (predicts next word, again and again)
- **Pre-trained**: trained once on enormous data, then usable for many tasks
- **Transformer**: uses the attention mechanism

6.4 The GPT Timeline — A Story of Scale

Model	Year	Parameters	What changed
GPT-1	2018	117 million	Proof of concept: pre-training works
GPT-2	2019	1.5 billion	Writes coherent paragraphs — OpenAI feared misuse
GPT-3	2020	175 billion	Few-shot learning: does tasks with just examples in prompt
GPT-4	2023	∼1.8 trillion	Near-human reasoning, multimodal (text + images)

The pattern: each generation was roughly 10× bigger. And each jump in scale produced **emergent abilities** — capabilities that appeared "spontaneously" just from having more parameters and more data.

Analogy

GPT-2 (2019) was like a gifted student who can write decent essays.
GPT-3 (2020) was like that student graduating university — suddenly capable of translation, code, poetry, reasoning.
GPT-4 (2023) is like that student becoming a polymath — it handles text, images, logic, math, and code at near-expert level.
Each jump came not from teaching new tricks, but simply from *making the model bigger and feeding it more data.*

6.5 Why "Predict the Next Word" is So Powerful

This seems almost too simple. The model is just trained to predict what word comes next. But to predict well, the model must implicitly learn:

- Grammar and syntax
- Facts about the world

- Logic and reasoning
- Cause and effect
- How to code, translate, summarize, answer questions...

All of this emerges from one objective: **predict the next token**. It's the simplest training objective that produces the most complex behavior.

6.6 BERT vs. GPT — Two Transformer Strategies

	BERT (Google, 2018)	**GPT (OpenAI, 2018+)**
Direction	Reads text in both directions	Reads left-to-right only
Task	Fill in missing words (understanding)	Predict next word (generation)
Best for	Search, classification, Q&A extraction	Text generation, chatbots, creation
Products	Google Search	ChatGPT, GitHub Copilot

Chapter 7

Generative AI — Machines That Create

7.1 What is Generative AI?

Generative AI is the branch of AI that **creates new content** — text, images, code, music, video — that didn't exist before.

Before Generative AI, machines could only **analyze** (classify, detect, measure). Now they can **produce**:

Type of content	Tool	How it works
Text	ChatGPT, Claude, Gemini	Transformer predicts next word
Images	DALL-E, Midjourney, Stable Diffusion	Neural net generates pixels from description
Code	GitHub Copilot, Cursor	LLM trained on code repositories
Music	Suno, Udio	Neural net generates audio waveforms
Video	Sora, Runway	Frame-by-frame generation from text

Analogy

Traditional AI = a camera (it captures and analyzes what exists).
Generative AI = a painter (it creates something new that never existed before).
Both are impressive, but only the painter produces original content.

7.2 How Generative AI Works (Simplified)

All generative models follow the same fundamental pattern:

1. **Learn the structure** of the training data (what a sentence looks like, what a face looks like, what code looks like)
2. **Given a prompt** (instruction or starting point), generate new content that **follows the same structure**
3. **Output** looks like it could be real — but was created from scratch

For text (GPT), this means: having seen trillions of words, the model knows what "good text" looks like. When you ask a question, it generates an answer word-by-word, each word chosen to be the most likely continuation.

7.3 The Generative AI Ecosystem (2024–2026)

Category	Key players	Our relevance
Foundation models	OpenAI, Anthropic, Google, Meta	We use Azure OpenAI (GPT)
Enterprise platforms	Microsoft (Azure), AWS (Bedrock)	ST ChatGPT runs on Azure
Coding assistants	GitHub Copilot, Cursor	Used for pipeline development
Image generation	DALL-E, Midjourney	Not used in our project
Specialized agents	Custom deployments	Our preprocessing agent

7.4 Why Generative AI Changed Everything

Before 2022, AI was mostly invisible to non-specialists. Since ChatGPT (November 2022), everyone can interact with AI directly. This changed:

- **Accessibility**: anyone can use AI without coding
- **Productivity**: writing, coding, research became 2–10× faster
- **Expectations**: people now expect AI-powered features everywhere
- **Enterprise adoption**: companies rushed to integrate AI into workflows

But generative AI has a critical weakness: it can generate **confident nonsense** (hallucination). This is exactly why technologies like RAG (Chapter 11) exist — to ground generative AI in verified facts.

Chapter 8

Large Language Models — The Engines of Modern AI

8.1 What is an LLM?

A Large Language Model is a Deep Learning model (specifically a Transformer) trained on enormous amounts of text. It learns to predict what comes next, and from this develops the ability to understand and generate language.

"Large" refers to both:

- The **model size**: billions to trillions of parameters (weights)
- The **training data**: trillions of words from books, websites, code, conversations

Analogy

An LLM is like someone who has read every book, every website, and every forum in the world. They can talk about anything convincingly — but they might mix things up or make things up, because they don't truly "understand" — they just know what words typically follow other words.

8.2 How LLMs Generate Text

LLMs produce text **one token at a time**. For each position, the model calculates the probability of every possible next word and picks one:

```
Input: "How do I initialize" -> predicts: "UART"
Then: "How do I initialize UART" -> predicts: "on"
Then: "How do I initialize UART on" -> predicts: "STM32H7"
...continues until the answer is complete
```

The model never "plans" the full answer in advance. It generates word by word, always choosing the most probable continuation given everything that came before.

8.3 The Hallucination Problem

The most dangerous weakness of LLMs: they can generate information that sounds completely correct but is **entirely false**.

Hallucination example	Why dangerous
"This bug was fixed in v1.4.2"	Version might not exist — wrong firmware choice
"Use HAL_DMA_Start_IT() with channel 3"	Wrong channel = potential hardware damage
"According to the datasheet, max clock is 200MHz"	Wrong spec = design failure

Why it happens: the model generates the most *statistically likely* continuation, not the most *factually correct* one. It has no concept of truth — only of probability.

Solution: RAG (Chapter 11) — force the LLM to answer ONLY from retrieved, verified documents.

8.4 Key LLM Concepts

Concept	What it means
Prompt	The text you send (question + context + instructions)
Context window	Max text the model can process at once (e.g. 128K tokens for GPT-4)
Temperature	Controls randomness: 0 = deterministic, 1 = creative
System prompt	Hidden instructions defining behavior ("You are a helpful STM32 expert...")
Fine-tuning	Additional training on domain-specific data (expensive, permanent)
Tokens	Smallest text units (roughly: 1 token $\approx$ 0.75 words)

8.5 The LLM Landscape (2024–2026)

Model family	Company	Characteristics
GPT-4, GPT-4o	OpenAI	Most widely deployed, multimodal
Claude 3/4	Anthropic	Strong reasoning, safety-focused
Gemini	Google	Integrated with Google ecosystem
LLaMA 3	Meta	Open-weight, self-hostable
Mistral	Mistral AI (France)	Efficient, strong for its size

8.6 LLM Limitations — What They Cannot Do

- **No real-time knowledge**: training data has a cutoff date
- **No memory between conversations**: each chat starts fresh (unless engineered otherwise)
- **Cannot verify facts**: no access to ground truth during generation
- **Cannot reason reliably**: appears to reason but can make logical errors
- **Cannot learn from interaction**: weights are frozen after training

These limitations explain why **RAG, tools, and agents** are necessary: they compensate for what LLMs cannot do alone.

ST ChatGPT uses GPT (via Azure OpenAI) as its LLM. We don't train or fine-tune it — we feed it the right documents through RAG, and define its behavior through a system prompt (persona).

Chapter 9

Embeddings — How AI Understands Meaning

9.1 The Problem: Computers Only Understand Numbers

Computers cannot read text the way we do. They need text converted into **numbers** — but not random numbers. Numbers that capture the **meaning**.

9.2 What is an Embedding?

An embedding is a list of numbers (a vector) that represents the **meaning** of a piece of text. Texts with similar meanings get similar vectors.

"UART initialization error" and "USART init failure" would have nearly identical embeddings because they mean the same thing — even though they use completely different words.

Analogy

Think of GPS coordinates. Paris and Lyon are both in France, so their coordinates are close together. Tokyo is far away, so its coordinates are very different. **Embeddings work the same way but for meaning**: similar texts are "close" in embedding space, unrelated texts are "far apart."

9.3 Why Embeddings Matter for Search

Keyword search (TF-IDF)	Semantic search (embeddings)
Matches exact words	Matches meaning
"UART error" won't find "USART failure"	"UART error" WILL find "USART failure"
Simple but brittle	Robust to synonyms

9.4 Cosine Similarity

To measure how similar two embeddings are, we use **cosine similarity** — a number between 0 (completely different) and 1 (identical meaning):

Text A	Text B	Similarity
UART init error on H7	USART initialization failure STM32H7	0.92
UART init error on H7	How to configure SPI clock	0.15

9.5 Embeddings Are the Foundation of RAG

In a RAG system:

1. All documents are converted to embeddings and stored in a vector database
2. When a user asks a question, the question is also converted to an embedding
3. The system finds documents whose embeddings are closest to the question's embedding
4. Those documents are given to the LLM to generate the answer

Without good embeddings, RAG cannot work. This is why data quality matters: garbage text produces garbage embeddings, which means wrong documents are retrieved.

Chapter 10

Specialized Chatbots — Why Generic AI Isn't Enough

10.1 Generalist vs. Specialized

	Generalist (ChatGPT free)	**Specialized (our chatbot)**
Knowledge	Broad but shallow	Deep in one domain
Accuracy	Hallucination risk	Answers grounded in KB
Data source	The entire internet	Our curated documents
Updates	Waits for model retraining	KB updated anytime

10.2 Why Specialize?

A generalist LLM does **not** know:

- The specific bugs of HAL STM32H7 v1.11.0
- The exact DMA configuration for an SPI transfer
- Workarounds validated by the firmware team

You must **provide this information at query time** → that's the role of RAG.

10.3 Three Approaches to Specialization

Approach	**How**	**Pros**	**Cons**
Fine-tuning	Retrain on your data	Deeply integrated	Expensive, risks "forgetting"
RAG	Provide docs at query time	Flexible, traceable, updatable	Depends on retrieval quality
Prompt engineering	Detailed instructions	Quick to set up	Limited by context window

10.4 Why We Chose RAG

- Data changes constantly (new issues, firmware releases)
- Every answer must cite its source (traceability)
- Fine-tuning would require expensive retraining at every update
- RAG: update the Knowledge Base in minutes, model stays the same

Chapter 11

RAG — Retrieval-Augmented Generation

11.1 The Problem RAG Solves

Standard LLMs answer from their training data — which is frozen, generic, and may be wrong for your specific domain.

LLM alone	**LLM + RAG**
Answers from general training	Answers from YOUR documents
May be outdated	Uses your latest data
Cannot cite sources	Every answer traceable
High hallucination risk	Grounded in facts

11.2 How RAG Works — Step by Step

1. **User asks a question** — "How to configure DMA on STM32H7?"
2. **RETRIEVAL** — Search the Knowledge Base for the most similar documents (using embeddings)
3. **Top-k documents retrieved** — The most relevant chunks are selected
4. **AUGMENTED GENERATION** — LLM receives: question + retrieved docs
5. **Answer generated** — LLM synthesizes a precise, sourced answer

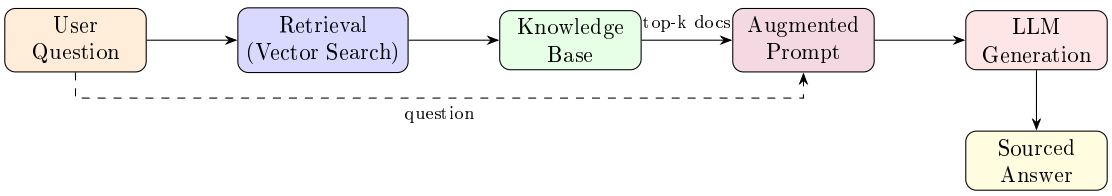


Figure 11.1: RAG flow: Retrieval → Augmentation → Generation.

11.3 Why Data Quality is CRITICAL for RAG

RAG can only work if:

1. The **right data exists** in the KB (Data Generation)
2. The data is **well structured** to be found (Feature Engineering + Chunking)
3. The **retrieval works** (Similarity + Evaluation)

→ This is the role of our pipeline: transform raw data into optimized data so RAG works.

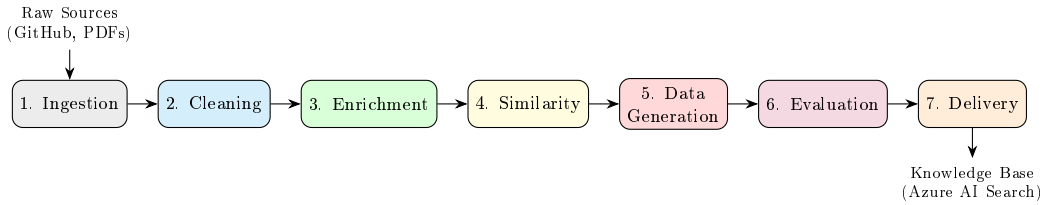


Figure 11.2: Our preprocessing pipeline: 7 stages from raw data to RAG-ready Knowledge Base.

11.4 Chunking Strategies

Documents must be split into "chunks" (pieces) for retrieval. Strategy matters:

Strategy	**How**	**Tradeoff**
Full document	1 chunk = whole doc	Simple but noisy
Paragraph	Split on paragraphs	Preserves meaning
Sliding window	Fixed size with overlap	Uniform but cuts meaning
Section-based	Split on headers	Great for structured docs
Parent-child	Children for search, parents for context	Best quality, complex

Chapter 12

Feature Engineering — Making Data Useful

12.1 What is Feature Engineering?

Feature Engineering is the art of **extracting and creating structured information** from raw data so that an AI system can work effectively.

Analogy

You have 5,000 paper CVs in a pile. Feature Engineering = reading each CV and filling a spreadsheet with columns: "Years of experience," "Main skill," "Education level." Now you can sort, filter, and search — impossible with raw papers.

12.2 Why It's Often More Impactful Than the Algorithm

A simple algorithm with **great features** outperforms a complex algorithm with **poor features** almost every time.

```
Raw text: "HAL_SPI_Transmit returns HAL_TIMEOUT on NUCLEO-H743ZI
          when DMA is used with buffer size > 65535 bytes"

Feature Engineering extracts:
  layer = "HAL" (from HAL_ prefix)
  component = "SPI" (from function name)
  board = "NUCLEO-H743ZI" (from board pattern)
  severity = "high" (timeout = functional failure)
  keywords = ["DMA", "buffer size", "timeout"]
```

12.3 Why It's Critical for RAG

Without features: RAG searches raw unstructured text → mediocre results.

With features: RAG can filter by layer, prioritize by severity, target by board → excellent results.

Our enrichment stage performs feature engineering: layer, severity, board, component, issue_kind, keywords, evidence strength — extracted from every issue and file.

Chapter 13

Agentic AI — Autonomous Intelligent Agents

13.1 What is an AI Agent?

An **AI agent** is an autonomous program that:

1. Receives an **objective** (not a simple instruction)
2. **Plans** steps to achieve it
3. **Executes** actions (APIs, file operations, decisions)
4. **Observes** results
5. **Adapts** if something fails

```
Simple LLM: Question -> Answer (one shot)
AI Agent: Objective -> Plan -> Act -> Observe -> Act -> ... -> Done
```

Analogy

Simple LLM = a consultant you ask ONE question.
AI Agent = an intern given a MISSION. They plan, execute, handle problems, and come back with the result — without asking for help at every step.

13.2 Components of an Agent

- **LLM (brain)**: reasoning and decision-making
- **Tools**: APIs, databases, file systems
- **Memory**: context of past actions
- **Loop**: Plan → Act → Observe → Adapt → Repeat

13.3 Types of Agents

13.4 In Our Project: The Image Enrichment Agent

Objective: "For each issue containing images, generate a textual description of each image."

Type	Behavior	Example
Reactive	Responds to stimuli, no planning	Simple chatbot
Deliberative	Plans before acting	Our image agent
Multi-agent	Several agents collaborating	One collects data, another evaluates

What the agent does autonomously:

1. Scans all issues with images
2. Extracts image URLs
3. Calls LLM Vision API to analyze each image
4. If image is inaccessible → handles error, moves to next
5. If rate limit hit → waits and retries
6. Inserts description into the document
7. Verifies coherence

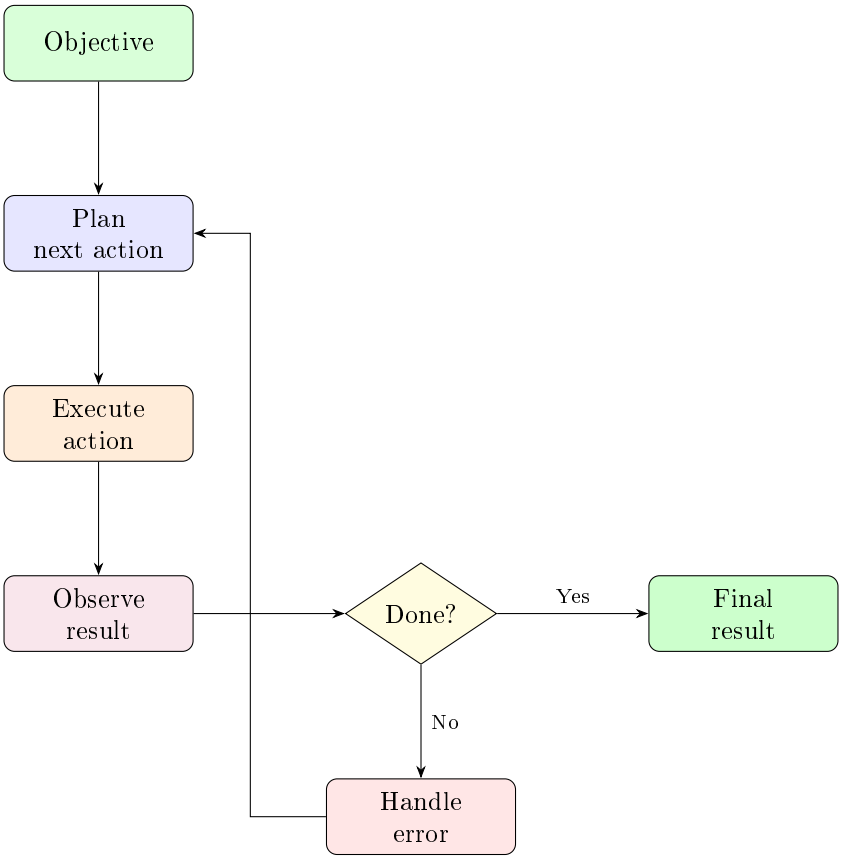


Figure 13.1: Agentic AI workflow: decide, act, observe, iterate.

13.5 In Our Project: The Preprocessing Pipeline Agent

The entire pipeline behaves as an **orchestration agent**:

Objective: "Transform raw GitHub data into a validated Knowledge Base, end-to-end, with no human in the loop."

Why this is agentic (not just a script):

- Makes **decisions**: skip invalid data, rescue borderline cases
- Handles **errors**: API limits, network failures, malformed data
- **Adapts**: new repositories added without code changes
- **Orchestrates**: GitHub API + LLM Vision + similarity engine + schema validator

Analogy

The Image Agent is a **specialist** (one task, done autonomously). The Pipeline Agent is a **project manager**: coordinates sub-processes, handles problems, delivers the final product without asking for help.

Chapter 14

Vector Databases — Where Meaning is Stored

14.1 What is a Vector Database?

A vector database stores embeddings and allows searching by **meaning** instead of keywords.

Traditional DB: "Find documents containing the word UART."

Vector DB: "Find documents *about* UART problems — even if they say 'serial communication failure'."

14.2 How Vector Search Works

1. All documents → converted to embeddings → stored
2. User query → converted to an embedding
3. Database finds the closest stored embeddings (cosine similarity)
4. Returns top-k most similar documents

14.3 The Landscape

Database	Type	Used for
Azure AI Search	Cloud managed	Our production system
ChromaDB	Open source, local	Our local testing
Pinecone	Cloud managed	Production RAG
FAISS	Library (Meta)	Research, high-performance

ST ChatGPT uses Azure AI Search as its vector database. We test locally with ChromaDB before uploading.

Chapter 15

Computer Vision — When AI Sees

15.1 What is Computer Vision?

Computer Vision enables machines to interpret images and videos — to "see" and extract information from visual content.

15.2 Key Tasks

Task	What it does	Example
Classification	Label an image	"Circuit diagram"
Object Detection	Locate objects	"Capacitor at position X,Y"
OCR	Extract text	Read register values from screenshot
Image Description	Generate text description	"Oscilloscope trace at 50MHz"

Our Vision Agent uses LLM Vision (Computer Vision + LLM) to describe technical images from STM32 issues, converting visual information into searchable text for RAG.

Chapter 16

Evaluation — Measuring AI Quality

16.1 Why Evaluation Matters

"If you cannot measure it, you cannot improve it." An AI system without evaluation is a car without a speedometer.

16.2 Key Metrics for RAG

Metric	What it measures	Perfect
P@k	Of top k results, how many are relevant?	1.0
MRR	How early is the first relevant result?	1.0
Recall	Of all relevant docs, how many did we find?	1.0
F1	Balance of precision and recall	1.0
Latency	Response speed	< 2s

16.3 Offline vs. Online Evaluation

Offline (local)	Online (production)
Test on your machine	Test on live platform
Fast, repeatable, free	Real conditions
Uses TF-IDF / local embeddings	Uses actual RAG engine
Catches data quality issues	Catches integration issues

16.4 The Iteration Loop

1. Modify pipeline (data, features, chunking)
2. Regenerate data
3. Evaluate — metrics improving?
    - YES → Deploy
    - NO → Back to step 1

We run offline TF-IDF tests on local JSONs + online 50-question benchmarks on the ST ChatGPT platform.

Chapter 17

Data Generation — Creating New Knowledge

17.1 What is Data Generation?

Data generation means **creating new data** that did not previously exist, using algorithms, rules, or AI models.

Unlike data *collection* (gathering what exists), data generation **produces** new documents from existing information.

Analogy

A teacher has 10 exam questions. To make a 100-question practice booklet, they write 90 new questions *inspired by* the originals — same topics, same difficulty, entirely new text. That's data generation: creating new content from existing knowledge.

17.2 Why Generate Data?

- **Insufficient data**: not enough examples
- **Imbalanced data**: too many of one category
- **Missing structure**: data exists but in unusable form
- **Expensive to collect**: human annotation costs time and money

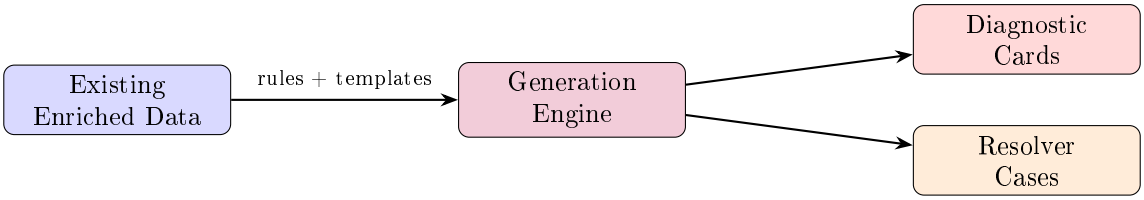
17.3 Types of Generated Data

Type	Description	Example
Synthetic	Created entirely by algorithms	GPT generating fake reviews for testing
Augmented	Transformations of existing data	Rotating images for more training samples
Derived	Synthesized from multiple sources	Bug reports → troubleshooting guide
Simulated	From simulating real scenarios	Virtual sensor data for autonomous driving

17.4 Derived Data: The Most Valuable Pattern

In enterprise AI, **derived data generation** takes existing structured data and synthesizes **entirely new documents** that provide more value than the originals:

- Multiple bug reports → a **diagnostic card** (root cause + symptoms + fix)
- A long issue thread → a **resolution guide** (step-by-step)
- Raw images → **text descriptions** (via Vision model)
- Individual issues → **similarity links** between related problems



*These are **new documents** that never existed in the raw data — synthesized from multiple sources.*

Figure 17.1: Derived data generation: existing data → new knowledge artifacts.

Analogy

A detective reads 200 witness statements and produces one "case summary" with timeline, suspects, and evidence links. The summary is *derived data* — it didn't exist before, was generated from existing sources, and is more useful than any single statement alone.

17.5 Quality Control

Generated data must be validated:

- **Schema validation**: required fields present?
- **Factual accuracy**: correctly reflects sources?
- **Deduplication**: no redundant artifacts?
- **Usefulness**: can the AI actually retrieve and use it?

Chapter 18

Advantages, Limits, and Challenges of AI

18.1 Advantages

- **Speed**: processes thousands of documents in seconds
- **Consistency**: same input = same processing every time
- **Scalability**: works on any volume once built
- **Pattern discovery**: finds connections humans miss
- **24/7**: never sleeps, never takes breaks

18.2 Limits

- **No common sense**: matches patterns, doesn't understand
- **Data dependency**: garbage in = garbage out
- **Hallucination**: generates confident nonsense
- **Bias**: if 90% of training data is about STM32H7, the model will underperform on STM32F4
- **Explainability**: hard to know WHY a specific answer was generated
- **Cost**: training large models = thousands of GPUs for weeks

18.3 Ethical Considerations

- **Privacy**: AI may process sensitive data
- **Fairness**: models must not discriminate
- **Transparency**: users should know they're talking to AI
- **Accountability**: who is responsible when AI is wrong?

Chapter 19

Glossary

Term	Definition
AI	Artificial Intelligence — systems performing tasks requiring human intelligence
ML	Machine Learning — learning from data instead of explicit rules
DL	Deep Learning — ML with multi-layer neural networks
NLP	Natural Language Processing — AI for understanding text
Transformer	Architecture using attention mechanism (basis of all modern LLMs)
Attention	Mechanism allowing model to look at all words simultaneously
GPT	Generative Pre-trained Transformer
Generative AI	AI that creates new content (text, images, code)
LLM	Large Language Model — Transformer trained on massive text
RAG	Retrieval-Augmented Generation — search then answer from your data
Embedding	Numerical vector representing the meaning of text
Vector DB	Database for searching embeddings by similarity
Cosine similarity	Measure of similarity between two vectors (0 to 1)
Hallucination	AI generating false information that sounds correct
Prompt	Input text sent to an LLM
Context window	Maximum text an LLM can process at once
Fine-tuning	Additional training on domain-specific data
Token	Smallest text unit processed by an LLM
Feature Engineering	Extracting structured info from raw data
Data Generation	Creating new data from existing sources
Agentic AI	AI systems that act autonomously with planning and tools
Pipeline	Chain of automated processing steps
Knowledge Base (KB)	Document collection feeding a RAG system
Chunking	Splitting documents into retrievable pieces
TF-IDF	Term Frequency-Inverse Document Frequency
P@k	Precision at rank k
MRR	Mean Reciprocal Rank
Epoch	One complete pass through training data

Term	Definition
Overfitting	Model memorizes training data, fails on new data
Model drift	Model performance degrading over time

Chapter 20

Conclusion

The Story in One Page

This document told a story with a natural progression:

1. **AI** is software that finds patterns — not magic
2. **Machine Learning** taught machines to learn from data instead of rules
3. **Deep Learning** made ML powerful enough for language and vision
4. **NLP** applied DL to text — machines started "reading"
5. **Transformers** (attention mechanism) solved NLP's speed and memory problems
6. **GPT** was born: a Transformer trained to predict next words became shockingly capable
7. **Generative AI** emerged: machines that create text, images, code
8. **LLMs** became the engines of modern AI — but they hallucinate
9. **RAG** solved hallucination by grounding LLMs in verified documents
10. **Feature Engineering** makes those documents findable
11. **Agents** automate the entire workflow end-to-end
12. **Data quality drives everything** — better data = better AI

The Bottom Line

AI amplifies human capability. It doesn't replace expertise — it makes experts faster.

Our project demonstrates this: we don't build the LLM, we don't train the neural network. We **engineer the data** that makes the AI useful. The chatbot's answer quality depends entirely on the data we provide — and that's what our pipeline ensures.